

Software Engineering

Projektdokumentation

Projektarbeit: Entwicklung einer Temperatursteuerung eines Backofens

Professor: Prof. Becker

Autor:

Lukas Kraft, 22206152

03. Juni 2025

Inhaltsverzeichnis

Einleitung

Im Rahmen der Projektarbeit im Fach "Software Engineering" bestand die Aufgabe darin, ein spezifisches Teilmodul eines größeren Softwaresystems eigenständig zu konzipieren und zu implementieren. Dabei lag der Fokus auf der vollständigen Durchführung aller wesentlichen Phasen des Softwareentwicklungsprozesses – von der Anforderungserhebung über Architektur und Design bis hin zur Umsetzung und Testung.

Das individuelle Thema dieser Arbeit war die Entwicklung einer Steuerung zur Temperaturregelung eines Backofens. Ziel war es, ein System zu entwerfen, das dem Benutzer die Auswahl verschiedener Heizmodi sowie eine präzise Temperatureinstellung ermöglicht und dabei sicherheitsrelevante Aspekte berücksichtigt. Die Steuerung sollte außerdem hardwareunabhängig simuliert und auf einem Standard-PC lauffähig umgesetzt werden. Als Benutzerschnittstelle standen zwei simulierte Drehregler (für Temperatur und Modus) sowie ein Display zur Anzeige von Temperatur, Status und Warnhinweisen zur Verfügung.

Die Umsetzung folgte einem iterativen Vorgehensmodell mit vier funktional abgeschlossenen Entwicklungszyklen. Nach jeder Iteration wurde eine lauffähige Version des Systems bereitgestellt, um schrittweise Funktionalität, Sicherheit und Benutzerfreundlichkeit zu erweitern. Im Verlauf des Projekts wurde besonderer Wert auf Modularität, Wiederverwendbarkeit, Nachvollziehbarkeit der Anforderungen und eine klare Trennung von Verantwortlichkeiten gelegt.

Diese Dokumentation beschreibt im Folgenden die eingesetzten Technologien, die gewählte Architektur, das Softwaredesign, die umgesetzten Anforderungen sowie die einzelnen Iterationen inklusive ihrer Ergebnisse und Testabdeckungen.

1 Entwicklungsumgebung

Für die Umsetzung des Projekts wurde eine moderne und leistungsfähige Entwicklungsumgebung gewählt, um eine strukturierte, nachvollziehbare und effiziente Softwareentwicklung zu ermöglichen. Die eingesetzten Werkzeuge und Technologien im Überblick:

- **Programmiersprache:** Die Anwendung wurde in **C#** entwickelt – einer objektorientierten Programmiersprache mit starker Integration in das .NET-Ökosystem. Die Sprache zeichnet sich durch eine klare Syntax, hohe Erweiterbarkeit durch NuGet-Pakete und gute Performance aus.
- **Entwicklungsumgebung (IDE):** Zum Einsatz kam **JetBrains Rider**, eine moderne IDE mit umfassender Unterstützung für .NET-Projekte. Sie bietet integriertes Debugging, Refactoring-Tools sowie eine enge Anbindung an Versionskontrollsysteme und Build-Werkzeuge.
- **Testing-Framework:** Für automatisierte Komponententests wurde **xUnit** verwendet. Das Framework ermöglichte eine kontinuierliche Überprüfung und Validierung einzelner Programmteile.
- **Diagramme und Modellierung:** Zur Erstellung von UML-Diagrammen (z. B. Klassen- und Aktivitätsdiagramme) wurde **draw.io** genutzt. Das Tool erleichtert die Visualisierung von Softwarearchitekturen und bietet zahlreiche Vorlagen zur effizienten Modellierung.
- **Dokumentation:** Die gesamte Projektdokumentation wurde in **L^AT_EX** verfasst, um ein konsistentes Layout, automatische Verzeichnisse sowie klar strukturierte Querverweise zu gewährleisten.
- **Versionsverwaltung:** Die Quellcodeverwaltung erfolgte über **GitHub**. Branches, Pull Requests und Commit-Historien ermöglichten eine transparente, kollaborative Entwicklung.
- **Aufgabenmanagement:** Für die Aufgabenplanung und Sprintverwaltung wurde **Jira** eingesetzt. Das Tool ermöglichte eine klare Zuweisung von Arbeitspaketen, Terminüberwachung und Fortschrittskontrolle.

2 Vorgehen

Für die Umsetzung des Projekts wurde ein strukturierter Entwicklungsprozess gewählt, der eine Kombination aus dem klassischen Wasserfallmodell und iterativen Elementen aus dem Scrum-Ansatz darstellt. Während Analyse, Design und Planung weitgehend linear durchgeführt wurden, erfolgte die eigentliche Implementierung inkrementell in mehreren Iterationen. So konnte nach jeder Phase eine lauffähige, getestete Version der Software bereitgestellt werden.

1. **Requirements Engineering:** Zu Beginn wurden sämtliche funktionalen, nicht-funktionalen und sicherheitsrelevanten Anforderungen systematisch erfasst. Anschließend wurden diese Anforderungen priorisiert, um die Umsetzung auf die wichtigsten Kernfunktionen auszurichten.
2. **Softwarearchitektur:** Auf Basis der Anforderungen wurde die Systemarchitektur entworfen. Dabei wurden die Hauptmodule, ihre Schnittstellen sowie die zentralen Datenflüsse definiert. Ziel war eine modulare, erweiterbare und wartbare Struktur als Grundlage für die Implementierung.
3. **Softwaredesign:** Aufbauend auf der Architektur wurden die einzelnen Komponenten detailliert spezifiziert. UML-Diagramme wie Klassendiagramme oder Zustandsdiagramme visualisierten dabei Strukturen und Abläufe. Im Fokus standen die klare Trennung von Verantwortlichkeiten, hohe Wiederverwendbarkeit und Erweiterbarkeit.
4. **Implementierung und Test:** Die Umsetzung erfolgte in vier aufeinander aufbauenden Iterationen. Jede Iteration beinhaltete die Implementierung neuer Funktionalitäten sowie zugehöriger automatisierter Tests mit `xUnit`. Nach jeder Iteration stand eine lauffähige Version zur Verfügung, die anschließend erweitert und optimiert wurde. Dieses Vorgehen ermöglichte eine kontinuierliche Integration und Verbesserung des Gesamtsystems.

3 Annahmen über das System

Die nachfolgenden Anforderungen basieren auf spezifischen Annahmen über die Systemumgebung und verfügbare Hardwarekomponenten. Diese Annahmen bilden die Grundlage für das Design, die Funktionalität und die Umsetzung der Steuerung:

- **Display:** Das System verfügt über ein integriertes Display, das folgende Informationen darstellen kann:
 - Aktuelle Temperatur im Garraum
 - Symbol zur Anzeige des Vorheizstatus
 - Uhrzeit bzw. verbleibende Laufzeit eines Timers
 - Warnhinweis bei automatischer Abschaltung
- **Bedienelemente:** Die Steuerung erfolgt über zwei physische Drehregler:
 - Ein Regler zur Auswahl des Heizmodus (Betriebsart)
 - Ein Regler zur Temperatureinstellung mit Einrastfunktion zur sicheren Moduswahl
- **Heizelemente:** Der Backofen besitzt drei unabhängig ansteuerbare Heizaggregate:
 - Oberer Heizkörper
 - Unterer Heizkörper
 - Ringheizkörper an der Rückseite
- **Lüftung:** Ein Ventilator an der Rückwand kann separat ein- oder ausgeschaltet werden.
- **Sensorik:**
 - Ein Türkontaktsensor erkennt, ob die Backofentür geöffnet oder geschlossen ist.
 - Ein Temperaturfühler misst kontinuierlich die aktuelle Garraumtemperatur.
- **Schaltverhalten:** Sämtliche Heizaggregate und der Ventilator lassen sich ausschließlich binär (Ein/Aus) steuern.

4 Requirements Engineering

4.1 Funktionale Anforderungen

- R-1.1** Das System muss es dem Benutzer ermöglichen, die Temperatur über einen Drehregler einzustellen.
- R-1.2** Das System muss Temperatureinstellungen in 1 °C-Schritten zulassen.
- R-1.3** Das System muss Temperatureinstellungen im Bereich von 50 °C bis 300 °C unterstützen.
- R-1.4** Das System muss dem Benutzer erlauben, eine Betriebsart über einen separaten Drehregler auszuwählen.
- R-1.5** Folgende Betriebsmodi müssen wählbar sein:
- Ober-/Unterhitze
 - Oberhitze
 - Unterhitze
 - Grillfunktion
 - Umluft
 - Heißluft
- R-1.6** Das System muss die Heizaggregate entsprechend der gewählten Betriebsart automatisch aktivieren:
- Ober-/Unterhitze: Oberer und unterer Heizkörper
 - Oberhitze: Oberer Heizkörper
 - Unterhitze: Unterer Heizkörper
 - Grillfunktion: Oberer Heizkörper
 - Umluft: Oberer und unterer Heizkörper + Ventilator
 - Heißluft: Ringheizkörper hinten + Ventilator
- R-1.7** Das System muss die Heizaggregate einzeln ansteuern können.
- R-1.8** Das System muss die Temperaturregelung mit einer Abtastrate von mindestens 1 Hz durchführen.
- R-1.9** Das System muss alle Heizaggregate deaktivieren, wenn die aktuelle Temperatur gleich oder größer der Solltemperatur ist.
- R-1.10** Das System muss die gemäß Betriebsart definierten Heizaggregate aktivieren, wenn die aktuelle Temperatur unterhalb der Solltemperatur liegt.
- R-1.11** Im Grillmodus muss das System die eingestellte Temperatur intern in vier Grillstufen umwandeln:
- Stufe 1: 240 °C
 - Stufe 2: 260 °C
 - Stufe 3: 280 °C

- Stufe 4: 300 °C

R-1.12 Das System muss dem Benutzer folgende Informationen über ein Display anzeigen:

- Aktuelle Temperatur im Garraum
- Vorheizstatus (z. B. Symbol, wenn Solltemperatur erreicht)
- Restzeit eines eingestellten Timers
- Warnung bei automatischer Abschaltung

4.2 Sicherheitsanforderungen

R-2.1 Das System muss den Heizbetrieb automatisch abschalten, wenn die Temperatur 320 °C überschreitet.

R-2.2 Das System darf nicht weiter Heizen, wenn die Backofentür geöffnet ist.

R-2.3 Das System muss eine Fehlfunktion eines Heizaggregats erkennen, wenn die Temperatur innerhalb von 10 Abtastzyklen um weniger als 1 °C steigt. In diesem Fall muss eine Warnung ausgegeben oder der Fehler im Systemprotokoll erfasst werden.

4.3 Nicht-funktionale Anforderungen

R-3.1 Das System muss in der Lage sein, die Solltemperatur von 200 °C innerhalb von maximal 10 Minuten zu erreichen.

R-3.2 Das Display muss Statusänderungen innerhalb von maximal 1 Sekunde anzeigen.

R-3.3 Das System muss alle sicherheitsrelevanten Zustandsänderungen und Fehlerereignisse mit Zeitstempel in der Konsole ausgeben.

4.4 Priorisierung

ID	Requirement	Priorität
R-1.1	Temperatur über Drehregler einstellbar	1
R-1.2	Temperatureinstellung in 1 °C-Schritten	1
R-1.3	Temperaturbereich 50–300 °C	1
R-1.4	Betriebsart über Drehregler auswählbar	2
R-1.5	Auswahl definierter Betriebsmodi	2
R-1.6	Heizaggregate je nach Betriebsart aktivieren	2
R-1.7	Heizaggregate einzeln ansteuerbar	1
R-1.8	Temperaturregelung mit mind. 1 Hz	2
R-1.9	Heizabschaltung bei Solltemperatur erreicht	1
R-1.10	Heizaktivierung bei Temperatur unter Sollwert	1
R-1.11	Grillstufenumrechnung	2
R-1.12	Anzeige von Temperatur, Status, Timer, Warnungen	1
R-2.1	Abschaltung bei >320 °C	3
R-2.2	Kein Heizen bei geöffneter Tür	3
R-2.3	Fehlererkennung Heizaggregat	3
R-3.1	200 °C in 10 Minuten erreichen	2
R-3.2	Statusanzeige innerhalb 1 Sekunde	2
R-3.3	Fehlerprotokoll mit Zeitstempel	3

5 Softwarearchitektur

In diesem Kapitel werden alle zentralen Module des Systems beschrieben und ihre jeweilige Funktion erläutert. Die strukturellen Abhängigkeiten zwischen den Modulen sind in Abbildung 1 dargestellt.

Das Architekturkonzept verfolgt das Ziel, sämtliche Systemaufgaben in klar abgegrenzte Module zu unterteilen. Dabei werden alle Hardwarekomponenten (z. B. Sensoren, Heizkörper) sowie übertragene Daten in separaten Klassen gekapselt. Dies unterstützt eine saubere Trennung der Verantwortlichkeiten und erleichtert die Wartung und Erweiterung des Systems. Zentrale Datenstrukturen werden im Modul `GlobalModels` definiert und systemweit verwendet. Zusätzlich wird ein dediziertes `LoggingModule` eingesetzt, um alle Protokollierungsaufgaben vom Anwendungscode zu trennen.

Main

Das Modul `Main` dient als Einstiegspunkt des Programms. Es initialisiert alle notwendigen Komponenten und erstellt eine Instanz des zentralen Steuerungsmoduls `OvenController`.

OvenControllerModule

Der `OvenController` stellt die zentrale Steuereinheit des Systems dar. Er koordiniert die Funktionsmodule innerhalb einer fortlaufenden Hauptschleife. Zu seinen Aufgaben zählen:

- Erfassen der Benutzereingaben über das `InputHandlerModule`
- Weitergabe der Eingaben an das `ModeControlModule` zur Interpretation
- Ausgabe von Informationen über das `OutputHandlerModule`
- Start und Überwachung des `SafetyModule` in einem separaten Thread

Die Ablauflogik der Steuerungsschleife ist in Abbildung 2 dargestellt.

SafetyModule

Das `SafetyModule` führt kontinuierlich Sicherheitsüberprüfungen durch und läuft unabhängig vom Hauptprozess in einem eigenen Thread. Dadurch werden kritische Systemzustände (z. B. Überhitzung oder geöffnete Tür) frühzeitig erkannt und entsprechende Maßnahmen eingeleitet.

InputHandlerModule

Dieses Modul verarbeitet alle Benutzereingaben. Es liest die Einstellungen der beiden Drehregler (Temperatur und Betriebsmodus) sowie den Timerwert aus. Die gesammelten Eingaben werden als strukturierte Objekte an den `OvenController` übergeben.

ModeControlModule

Das `ModeControlModule` interpretiert die übergebenen Eingaben und entscheidet, welche Heizaggregate in Abhängigkeit vom gewählten Modus aktiviert werden sollen. Die konkrete Steuerung erfolgt über das `ThermalControlModule`.

ThermalControlModule

Dieses Modul enthält die Ansteuerlogik für alle Heizaggregate und den Ventilator. Es stellt die direkte Schnittstelle zur (simulierten) Hardware dar und aktiviert oder deaktiviert Komponenten gemäß den Anforderungen des `ModeControlModule`.

OutputHandlerModule

Das `OutputHandlerModule` ist für die Darstellung aller relevanten Informationen auf dem Display zuständig. Dies umfasst:

- Aktuelle Temperatur im Garraum
- Gewählter Betriebsmodus
- Timerstatus
- Warnmeldungen und Sicherheitsanzeigen

SensorModule

Das `SensorModule` kapselt die simulierten Sensoren des Systems. Dazu gehören:

- `TemperatureSensor`: misst die aktuelle Temperatur im Garraum
- `DoorSensor`: erkennt den Zustand der Backofentür (offen/geschlossen)

GlobalModels

Im Modul `GlobalModels` sind alle zentralen Datenstrukturen definiert, die zur intermodularen Kommunikation und Zustandshaltung benötigt werden:

- `InputValues`: enthält Temperatur, Modus und Timerdaten
- `CookingMode`: beschreibt alle verfügbaren Betriebsmodi
- `OvenState`: bildet den aktuellen Zustand des Systems ab (z. B. `Idle`, `Preheating`, `Active`)

LoggingModule

Das `LoggingModule` ist für die zentrale Protokollierung aller sicherheitsrelevanten Ereignisse und Statusänderungen verantwortlich. Alle Module können dieses zur Ausgabe strukturierter Logeinträge mit Zeitstempel nutzen.

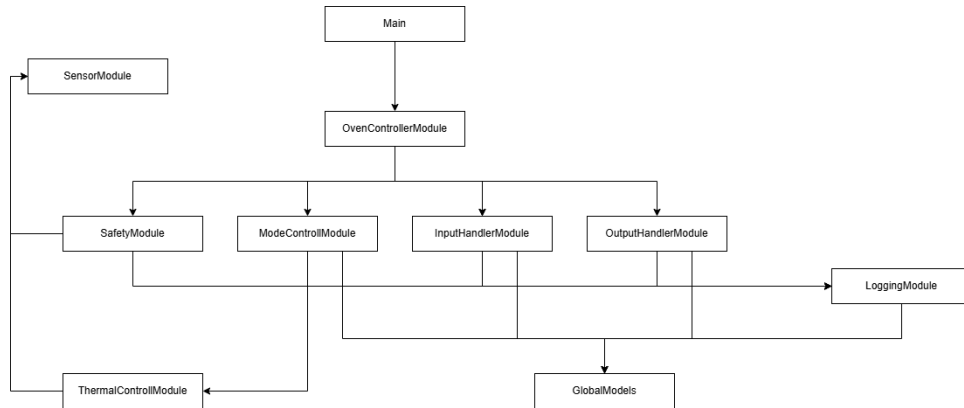


Abbildung 1: Modulübersicht des Systems

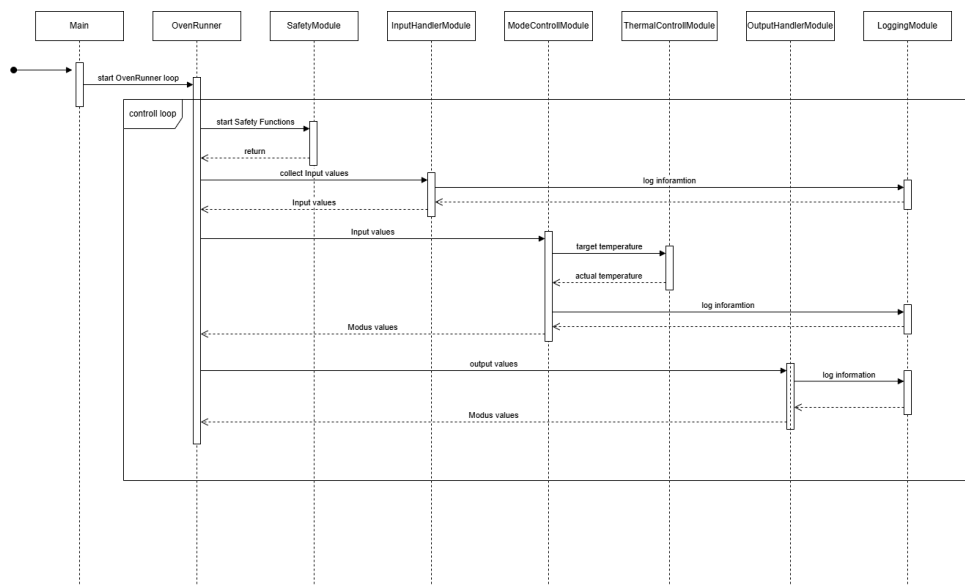


Abbildung 2: Kontrollschleife des OvenControllers (Iteration 1)

6 Software Design

Im Folgenden wird das Design der zentralen Module des Backofensystems beschrieben. Jedes Modul übernimmt eine klar definierte Aufgabe und orientiert sich an bewährten Software-Designprinzipien, insbesondere dem *Strategy Pattern*, *State Pattern* sowie der strikten Trennung zwischen Sensorik, Steuerung und Darstellung.

Designziele

Das Software-Design verfolgt folgende Hauptziele:

- **Erweiterbarkeit:** Neue Betriebsmodi, Funktionen oder Anzeigeelemente sollen mit minimalem Änderungsaufwand integrierbar sein. Dies wird durch konsequente Modularisierung und lose Kopplung erreicht.
- **Beherrschbare Komplexität:** Durch den Einsatz etablierter Entwurfsmuster – wie dem *Strategy Pattern* für Regelalgorithmen und dem *State Pattern* für Zustandsverwaltung – wird die Systemlogik übersichtlich und nachvollziehbar strukturiert.
- **Klare Verantwortungstrennung:** Die Architektur folgt einem Schichtenmodell mit den Bereichen *Sensorik* (Datenerfassung), *Steuerung* (Verarbeitung und Logik) und *Darstellung* (Benutzerausgabe). Jede Schicht ist funktional abgegrenzt, was Wartbarkeit und Erweiterbarkeit fördert.
- **Clean Code:** Der Quellcode ist klar strukturiert, konsistent dokumentiert und folgt gängigen Namenskonventionen. Das unterstützt die langfristige Pflege des Systems.
- **Trennung von Produktivcode und Logging:** Logging-Mechanismen sind vom Anwendungscode entkoppelt und in einem eigenen Modul gekapselt. Dadurch bleibt die Steuerlogik schlank und frei von Nebenfunktionen.
- **Testbarkeit:** Die lose Kopplung und klar definierten Schnittstellen zwischen Modulen erleichtern Unit-Tests. Durch die konsequente Entkopplung lassen sich Komponenten isoliert validieren.

Diese Designprinzipien stellen sicher, dass das System nicht nur funktional robust ist, sondern auch langfristig wartbar und flexibel weiterentwickelt werden kann.

GlobalModels

Das Modul `GlobalModels` definiert zentrale Datenstrukturen und Enumerationen, die als gemeinsame Kommunikationsschnittstelle zwischen den Modulen dienen. Die Übergabe von Informationen erfolgt über klar gekapselte Objekte, was die Erweiterbarkeit fördert und die Konsistenz der Datenhaltung sicherstellt.

Die Klasse `InputValues` aggregiert alle wesentlichen Eingaben des Benutzers:

- die gewünschte Zieltemperatur (Typ: `double`),
- den gewählten Heizmodus (Typ: `CookingMode`),
- sowie die eingestellte Timerdauer (Typ: `TimeSpan`).

Ergänzend dazu enthält die Klasse `OutputValues` alle Informationen, die dem Benutzer auf dem Display angezeigt werden sollen:

- die aktuelle Temperatur,
- ein Statusindikator zur Zieltemperatur,
- der aktuelle Timerstand,
- sowie ein Warnhinweis bei sicherheitsrelevanten Ereignissen.

Die Enumeration `CookingMode` beschreibt alle verfügbaren Heizmodi des Backofens, darunter z. B. `Grill`, `Umluft` oder `OberUnterhitze`.

Der Enum `OvenState` bildet die internen Zustände des Systems ab, darunter `Idle`, `Preheating`, `Active`, `Error` und `Off`.

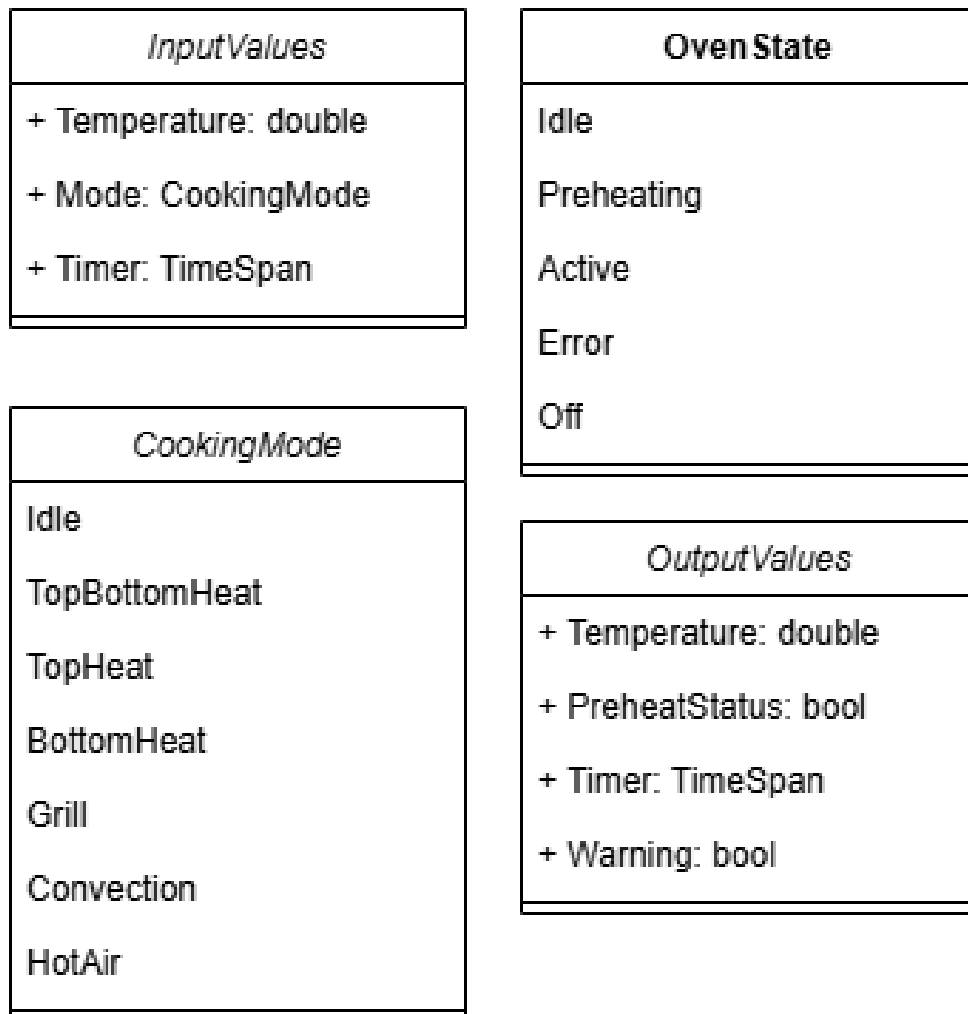


Abbildung 3: Klassendiagramm des Moduls GlobalModels

Main Modul

Das **Main**-Modul bildet den Einstiegspunkt der Anwendung. Es enthält die **Main()**-Methode, die beim Start des Programms automatisch ausgeführt wird.

In dieser Methode wird das Gesamtsystem initialisiert. Insbesondere wird eine Instanz des zentralen Steuerungsmoduls **OvenController** erstellt und dessen **Run()**-Methode aufgerufen, um den Betriebsablauf des Systems zu starten.

Das Modul übernimmt ausschließlich die Initialisierung und den einmaligen Systemstart. Weitere Logik ist im Sinne der Trennung von Verantwortlichkeiten nicht enthalten.

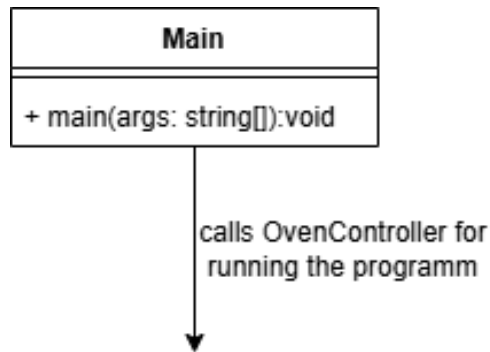


Abbildung 4: Klassendiagramm des **Main**-Moduls

OvenControllerModule

Das **OvenControllerModule** ist das zentrale Steuerungsmodul des Systems. Es implementiert die Ablaufsteuerung mithilfe des *State Pattern*, wobei der aktuelle Betriebszustand über eine Referenz auf ein entsprechendes Zustandsobjekt verwaltet wird.

In regelmäßigen Zyklen wird die Methode `Run()` aufgerufen. Innerhalb dieses Zyklus:

- werden die aktuellen Benutzereingaben über das **InputHandlerModule** erfasst,
- an das **ModeControlModule** zur Heizlogik-Interpretation übergeben,
- und die resultierenden Informationen über das **OutputHandlerModule** auf dem Display ausgegeben.

Zusätzlich wird in jeder Iteration der **TemperatureSensor** aktualisiert, um das Verhalten eines realen Sensors zu simulieren.

Die Zustandsverwaltung erfolgt über folgende implementierte State-Klassen:

- **IdleState** – Der Ofen ist inaktiv, sämtliche Heizungen sind deaktiviert.
- **PreHeatingState** – Der Ofen heizt aktiv auf die vorgegebene Zieltemperatur.
- **ActiveState** – Die Zieltemperatur ist erreicht und wird konstant gehalten.
- **ErrorState** – Es wurde ein sicherheitskritischer Zustand erkannt; alle Komponenten werden abgeschaltet. Rückkehr in einen anderen Zustand ist ausgeschlossen.
- **OffState** – Der Ofen wurde manuell oder durch Ablauf des Timers deaktiviert.

Das Modul reagiert asynchron auf Sicherheitsereignisse: Das **SafetyModule** kann den Controller unmittelbar in den **ErrorState** versetzen, um eine sofortige Abschaltung bei Gefährdung sicherzustellen.

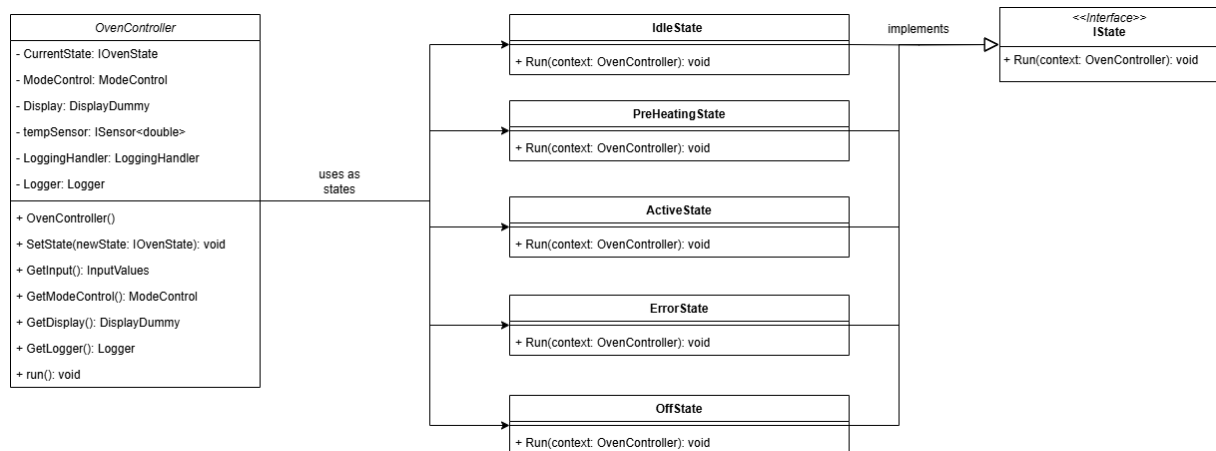
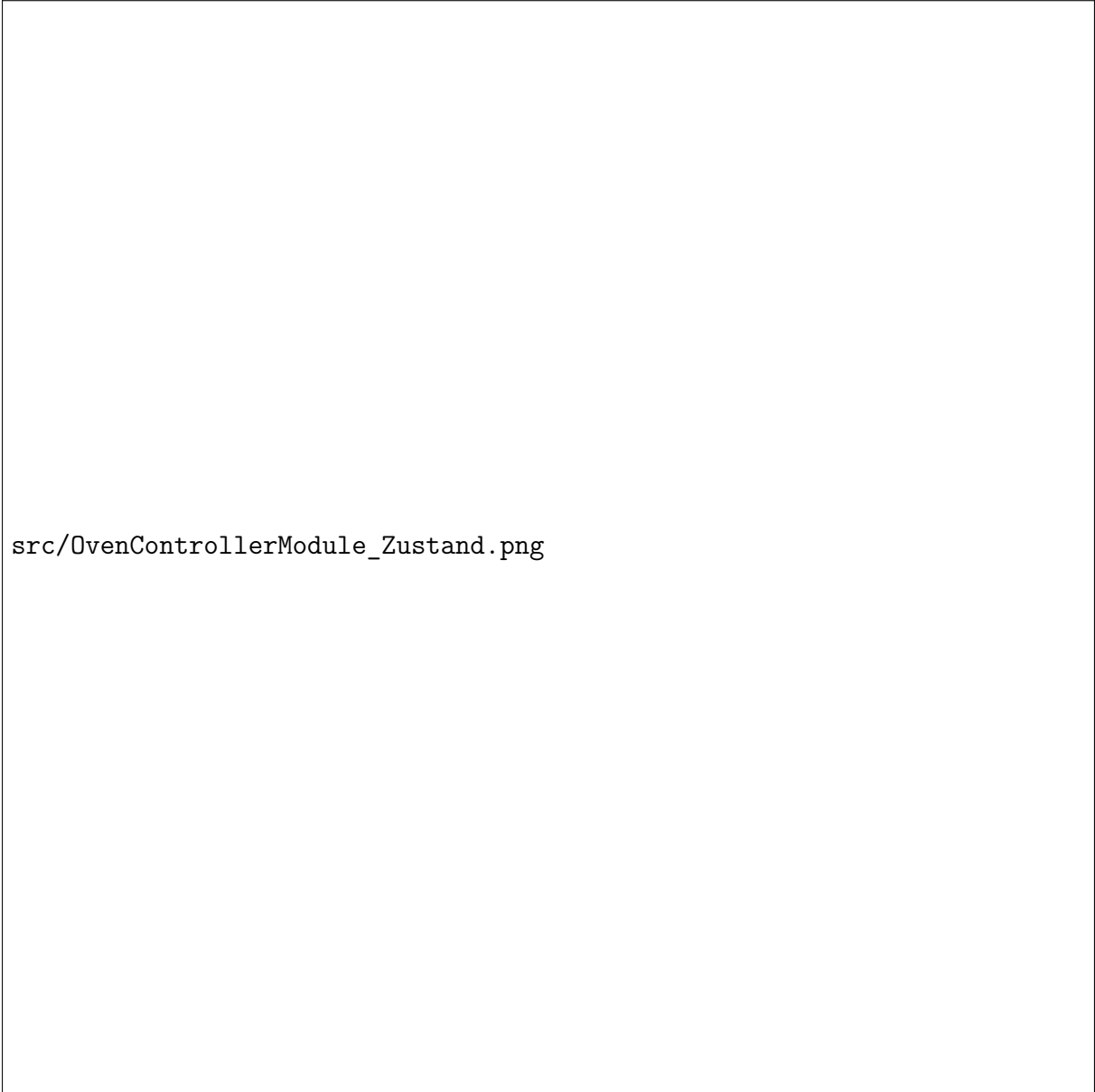


Abbildung 5: Klassendiagramm des **OvenControllerModule**



The image area is mostly blank, indicating that the state diagram content was not rendered. The text `src/OvenControllerModule_Zustand.png` is visible on the left side of the diagram area.

`src/OvenControllerModule_Zustand.png`

Abbildung 6: Zustandsdiagramm des `OvenControllerModule`

SafetyModule

Das **SafetyModule** ist vollständig vom übrigen System entkoppelt und wird in einem separaten Thread ausgeführt. Es überwacht kontinuierlich sicherheitsrelevante Systemzustände und reagiert im Fehlerfall unverzüglich.

Die zentrale Klasse **SafetyHandler** verwaltet eine Sammlung von Sicherheitsregeln. In festen Intervallen ruft sie deren **Check()**-Methoden auf. Bei einem Regelverstoß wird der **OvenController** unmittelbar benachrichtigt und in einen sicheren Zustand versetzt – entweder den **IdleState** oder **ErrorState**, je nach Schwere des Ereignisses.

Folgende Sicherheitsregeln sind aktuell implementiert:

- **OverheatRule**: Detektiert eine kritische Überschreitung der maximal zulässigen Temperatur und führt zum Wechsel in den **ErrorState**.
- **DoorOpenRule**: Erkennt eine geöffnete Tür während des Heizbetriebs. In diesem Fall wird in den **IdleState** gewechselt.
- **HeaterFailureRule**: Identifiziert defekte Heizaggregate, wenn trotz aktivierter Heizung über mehrere Zyklen kein Temperaturanstieg festgestellt wird. Dies führt zum Übergang in den **ErrorState**.

Durch die eigenständige Ausführung ist das **SafetyModule** in der Lage, unabhängig von Verzögerungen im Hauptprozess zuverlässig zu arbeiten. Damit trägt es wesentlich zur Betriebssicherheit und Fehlertoleranz des Systems bei.

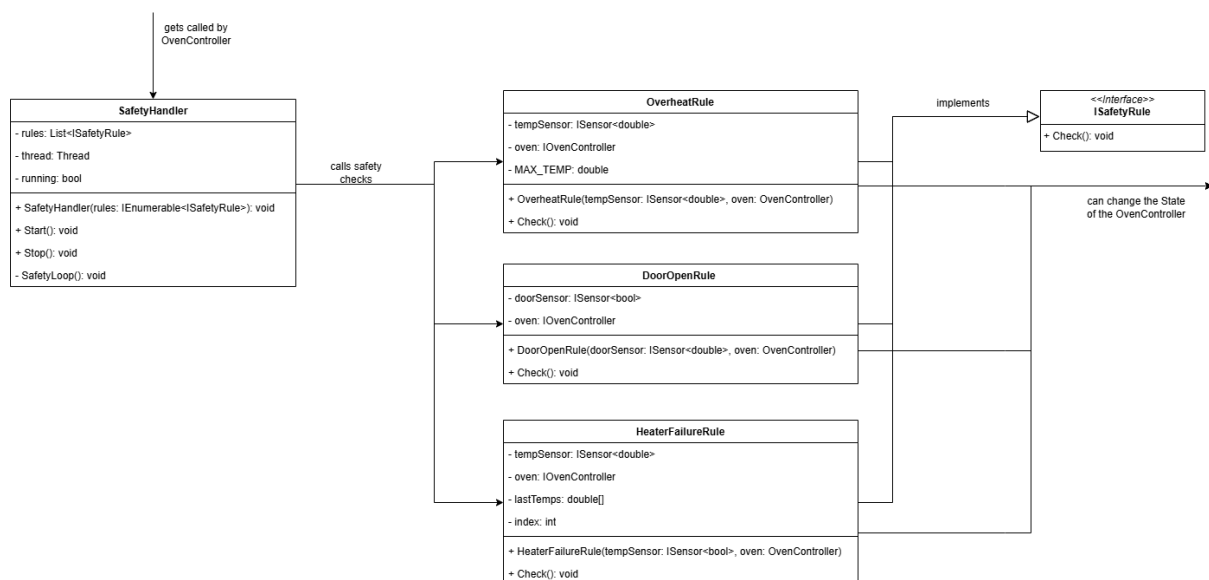


Abbildung 7: Klassendiagramm des **SafetyModule**

InputHandlerModule

Das `InputHandlerModule` ist für die Erfassung manueller Benutzereingaben zuständig. Es verarbeitet ausschließlich physische Interaktionen, nicht jedoch automatische Sensordaten.

Konkret werden die Drehwinkel zweier Regler ausgelesen:

- ein Regler zur Temperatureinstellung,
- ein Regler zur Auswahl des Betriebsmodus,
- sowie ein optionaler Timerwert.

Die Drehregler implementieren ein generisches Interface `IRotaryController<T>`, das unterschiedliche Rückgabetypen wie `int` (für Temperatur) oder `Enum` (für Moduswahl) unterstützt. Der aktuelle Winkel wird intern gespeichert und über die Methode `ReadInput()` in einen konkreten Wert übersetzt.

Die zentrale Klasse `InputHandler` bündelt alle Eingabewerte in einem Objekt vom Typ `InputValues`. Diese strukturierte Datenübergabe vereinfacht die Verarbeitung in nachgelagerten Modulen und unterstützt die Erweiterbarkeit.

Der Zugriff erfolgt über den `InputHandlerProxy`, der zusätzlich alle Eingaben protokolliert. Das zugrunde liegende Proxy-Pattern trennt dabei strikt zwischen Anwendungscode und Logging, wodurch die Hauptlogik schlank und wartbar bleibt.

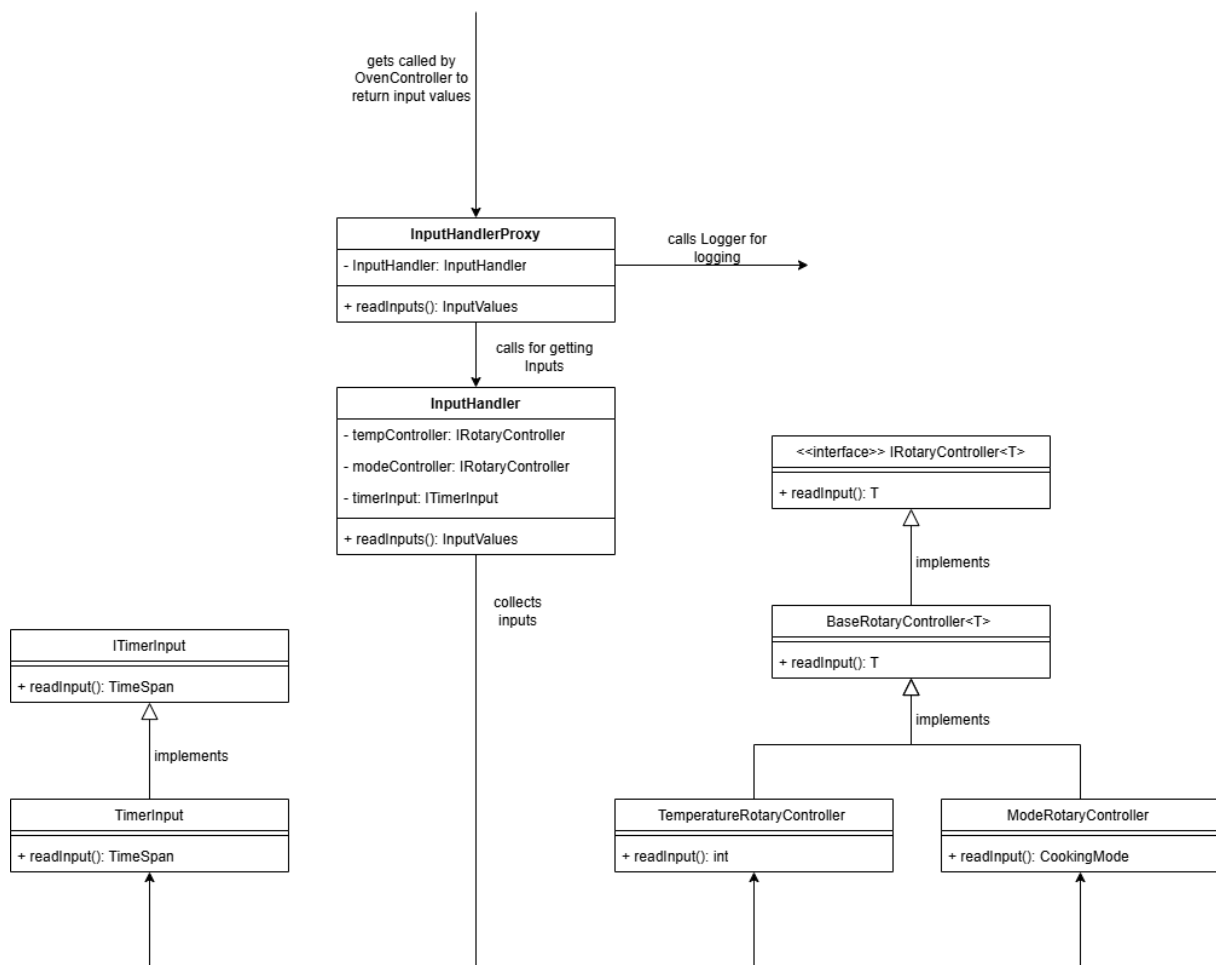


Abbildung 8: Klassendiagramm des InputHandlerModule

ModeControlModule

Das **ModeControlModule** implementiert die Heizlogik des Backofens nach dem *Strategy Pattern*. Die zentrale Klasse **ModeControl** verwaltet eine Referenz auf die aktuell aktive Strategie (**IModeStrategy**), die zur Laufzeit anhand des ausgewählten **CookingMode** ausgetauscht werden kann.

Die gewählte Strategie wird vom **OvenController** aufgerufen, um die entsprechenden Heizaggregate zu steuern. Dabei gibt jede Strategie einen Boolean-Wert zurück, der angibt, ob sich das System aktuell im Aufheizprozess befindet. Diese Information ist für den Zustand der State machine im **OvenControllerModule** relevant.

Folgende Heizstrategien sind verfügbar:

- **IdleMode**: keine Heizkomponenten aktiv (Standardzustand)
- **TopBottomHeatMode**: oberer und unterer Heizkörper aktiv
- **TopHeatMode**: nur oberer Heizkörper aktiv
- **BottomHeatMode**: nur unterer Heizkörper aktiv
- **GrillMode**: oberer Heizkörper aktiv, Zieltemperatur wird in Grillstufen übersetzt
- **CirculatingAirMode**: Ober- und Unterhitze kombiniert mit Ventilator
- **HotAirMode**: Ringheizkörper und Ventilator aktiv

Jede Strategie besitzt eine eigene Liste von **IThermalController**-Instanzen, über die die jeweiligen Heizkomponenten und der Ventilator gezielt angesteuert werden können. Der **GrillMode** bildet dabei eine Sonderform: Die eingestellte Zieltemperatur wird hier in definierte Grillstufen umgewandelt (z. B. 240–300°C).

Der Zugriff auf das Modul erfolgt über einen **ModeControlProxy**, der zusätzlich für die Protokollierung zuständig ist. Damit bleibt die Heizlogik vom Logging vollständig getrennt.

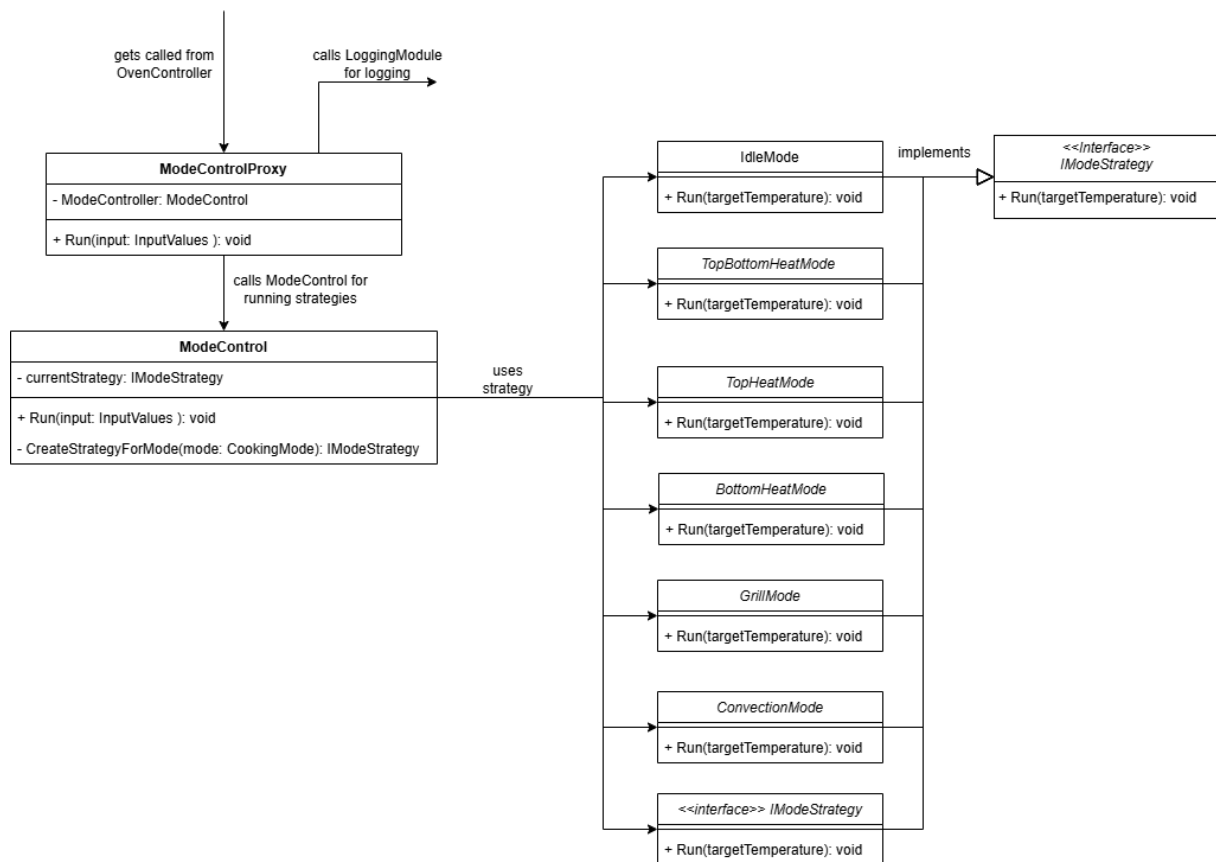


Abbildung 9: Klassendiagramm des ModeControlModule

ThermalControlModule

Das `ThermalControlModule` bildet die unterste Abstraktionsebene zur gezielten Ansteuerung der Heizkörper und des Ventilators. Jede Komponente ist als Singleton implementiert, um sicherzustellen, dass im System stets nur eine Instanz pro Aktor existiert.

Die beteiligten Klassen implementieren zwei zentrale Schnittstellen:

- `IThermalController` – zur Aktivierung und Deaktivierung der Komponente (Ein/Aus),
- `ITemperatureSource` – zur Bereitstellung von Temperaturwerten (sofern relevant).

Die Steuerung der Komponenten erfolgt ausschließlich durch Strategien im `ModeControlModule`, wodurch eine klare Trennung zwischen Logik und Ausführung gewährleistet ist.

Die folgenden Komponenten sind im Modul enthalten:

- `TopHeater` – oberer Heizkörper,
- `BottomHeater` – unterer Heizkörper,
- `RearHeater` – Ringheizkörper an der Rückseite,
- `Ventilator` – Umluftventilator.

Da die Steuerungslogik vollständig in den jeweiligen Strategien gekapselt ist, bleibt das `ThermalControlModule` selbst zustandslos, wartungsarm und gut testbar.

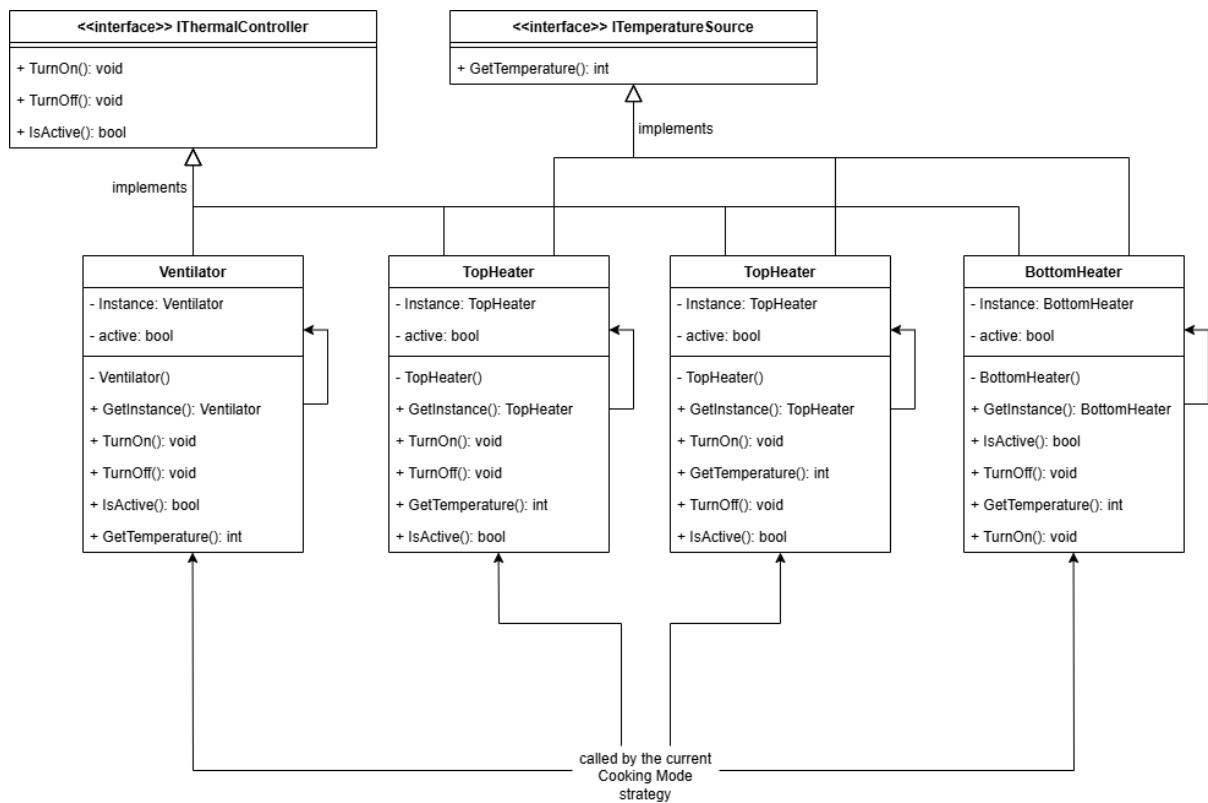


Abbildung 10: Klassendiagramm des ThermalControlModule

OutputHandlerModule

Das `OutputHandlerModule` übernimmt die Darstellung aller relevanten Systeminformationen auf einer simulierten Anzeige. Es bildet die logische Schnittstelle zu einem realen Display und sorgt für die visuelle Rückmeldung an den Benutzer.

Die zentrale Klasse `DisplayDummy` empfängt in regelmäßigen Intervallen ein Objekt vom Typ `OutputValues`, das sämtliche anzuzeigenden Informationen enthält. Dazu gehören:

- die aktuelle Temperatur im Garraum,
- der Status des Vorheizvorgangs (z. B. durch ein Symbol),
- die verbleibende Timerlaufzeit,
- sowie eine Warnanzeige bei sicherheitsrelevanten Zuständen.

Durch die Trennung von Darstellung und Logik bleibt das Modul klar abgegrenzt, erweiterbar und leicht austauschbar – beispielsweise bei einer späteren Integration eines echten Displays.

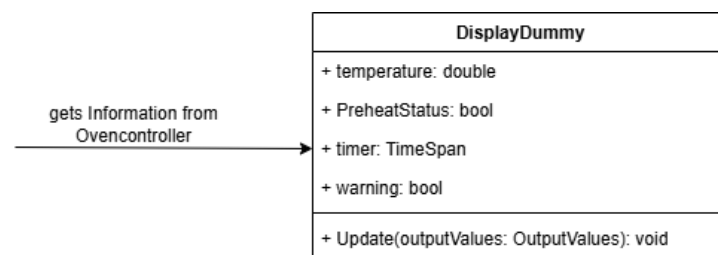


Abbildung 11: Klassendiagramm des `OutputHandlerModule`

SensorModule

Das **SensorModule** kapselt alle Sensoren, die unabhängig von Benutzereingaben automatisch Daten liefern. Jeder Sensor implementiert das generische Interface **ISensor<T>**, welches eine typisierte und standardisierte Abfrage der Sensordaten ermöglicht.

Aktuell sind zwei Sensoren im System implementiert:

- **TemperatureSensor**: Aggregiert die Temperaturinformationen aller aktiven Heizaggregate und liefert die aktuelle Garraumtemperatur.
- **DoorSensor**: Erkennt, ob die Backofentür geöffnet oder geschlossen ist, und meldet den Zustand binär.

Das Modul sorgt für eine saubere Trennung zwischen Sensorik und Logik. Die zentrale Definition der Sensoren erleichtert deren Austausch, Erweiterung oder Simulation in Testszenarien.

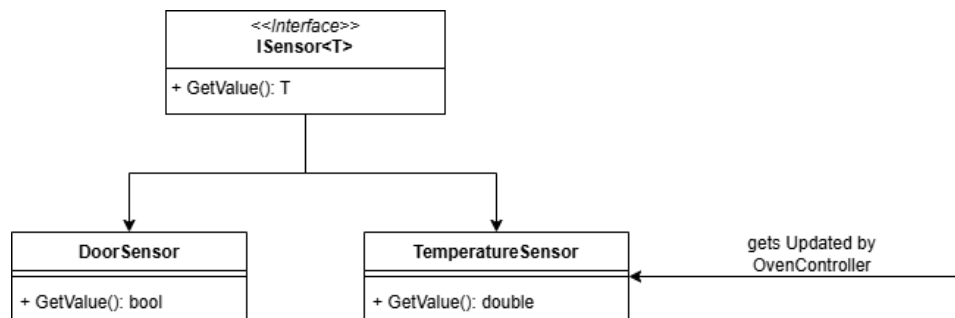


Abbildung 12: Klassendiagramm des **SensorModule**

LoggingModule

Das **LoggingModule** ist für die zentrale Protokollierung aller relevanten Systemereignisse verantwortlich. Es dient der Laufzeitüberwachung, Fehleranalyse und Nachvollziehbarkeit sicherheitsrelevanter Zustände.

Die zentrale Komponente des Moduls ist der **LoggingHandler**. Über ihn kann jeder Systemteil einen spezifischen Logger anfordern, der automatisch in einer zentralen Instanz verwaltet wird. Um sicherzustellen, dass genau eine gemeinsame Logging-Instanz im System existiert, ist der **LoggingHandler** nach dem Singleton-Prinzip implementiert.

Das Modul protokolliert u. a.:

- Benutzereingaben,
- Zustandswechsel der Statemachine,
- Warnungen und Fehlerereignisse.

Alle Logeinträge werden mit Zeitstempel versehen, wodurch eine konsistente und nachvollziehbare Historie der Systemaktivitäten gewährleistet ist.

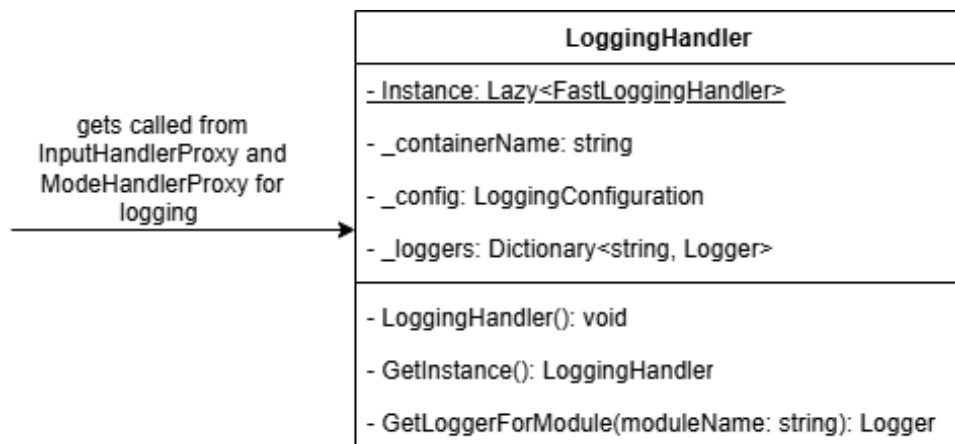


Abbildung 13: Klassendiagramm des LoggingModule

7 Geplante Implementierungsiterationen

Die Implementierung des Backofensystems erfolgt schrittweise in mehreren aufeinander aufbauenden Iterationen. Jede Iteration verfolgt ein klar abgegrenztes Entwicklungsziel und liefert eine lauffähige Teillösung. Auf diese Weise entsteht schrittweise eine modular aufgebaute und testbare Steuerungssoftware.

Iteration 1: Grundstruktur und Kernfunktionalität

In der ersten Iteration wird das Grundgerüst des Systems aufgebaut. Ziel ist es, eine erste funktionsfähige Version der Steuerung mit Temperaturregelung zu realisieren:

- Implementierung des `Main`-Moduls zur Initialisierung des Gesamtsystems.
- Erstellung der zentralen Datenklassen `InputValues` und `OutputValues` im `GlobalModels`-Modul.
- Umsetzung des `OvenControllerModule` mit einem initialen `ActiveState` zur Steuerung der Temperatur.
- Vollständige Implementierung des `InputHandlerModule` und `OutputHandlerModule` zur Benutzerschnittstelle.
- Erste Heizlogik im `ModeControlModule` mit Standardstrategie, die alle Heizaggregate aktiviert.
- Implementierung des `ThermalControlModule` mit allen Aktoren (Heizelemente und Ventilator).
- Umsetzung des `SensorModule` mit `TemperatureSensor` und `DoorSensor`.

Iteration 2: Logging und Architekturstrukturierung

In dieser Iteration wird die Protokollierung eingeführt, um Systemereignisse nachvollziehbar zu dokumentieren:

- Entwicklung des zentralen `LoggingModule` mit Singleton-Loggerstruktur.
- Einführung von Proxy-Klassen für das `InputHandlerModule` und `ModeControlModule`, um Logging vom Produktivcode zu entkoppeln.
- Strukturverbesserungen zur Vorbereitung auf erweiterte Funktionalitäten.

Iteration 3: Zustandslogik und Heizstrategien

In der dritten Iteration wird die vollständige Zustandsmaschine implementiert. Die Heizlogik wird um alle Betriebsmodi ergänzt:

- Erweiterung des `OvenControllerModule` um die Zustände `IdleState`, `PreHeatingState`, `ErrorState` und `OffState`.
- Umsetzung aller im System definierten Heizstrategien im `ModeControlModule`.
- Erweiterung der Proxy-Mechanismen für Zustandsübergänge (z. B. Logging bei Zustandswechseln).

Iteration 4: Sicherheitslogik und Timer

Die finale Iteration fokussiert sich auf die Absicherung des Systems durch Echtzeitüberwachung:

- Implementierung des `SafetyModule` als paralleler Thread mit zyklischer Regelprüfung.
- Umsetzung der Sicherheitsregeln:
 - `OverheatRule`: Überhitzungsschutz,
 - `DoorOpenRule`: Türkontrolle bei Heizbetrieb,
 - `HeaterFailureRule`: Erkennung fehlerhafter Heizaggregate.
- Ergänzung eines Timer-Features, das vom Benutzer gesetzt werden kann und die Restlaufzeit im Display anzeigt.

8 Iteration 1 – Grundfunktionen

8.1 Zeitraum

29.05.2025 – 30.05.2025

8.2 Zielsetzung

Ziel dieser ersten Iteration war die Umsetzung einer funktionsfähigen Grundversion des Systems mit Temperaturregelung. Dabei sollte es möglich sein, die gewünschte Temperatur über einen simulierten Drehregler einzustellen. Das System sollte die ausgewählten Heizaggregate sowie den Ventilator entsprechend aktivieren und die Solltemperatur zuverlässig erreichen und halten. Eine differenzierte Zustandsverwaltung oder komplexe Modusstrategien wurden in dieser Phase noch nicht berücksichtigt.

8.3 Umgesetzte Anforderungen

Im Rahmen dieser Iteration wurden die folgenden Anforderungen vollständig implementiert:

- **6.2** – Temperatureinstellung über Drehregler
- **6.2** – Einstellbare Temperatur in 1°C-Schritten
- **6.2** – Temperaturbereich von 50°C bis 300°C
- **6.2** – Einzelansteuerung der Heizaggregate
- **6.2** – Temperaturregelung mit 1 Hz Abtastrate
- **6.2** – Abschalten der Heizelemente bei Erreichen der Solltemperatur
- **6.2** – Aktivierung der Heizelemente bei Unterschreiten der Solltemperatur
- **6.2** – Anzeige von Temperatur, Vorheizstatus, Timer und Warnungen
- **6.2** – Erreichen von 200°C innerhalb von 10 Minuten

8.4 Traceability-Matrix

Requirement	Beschreibung	Klasse	Methode	Testfälle	Status
6.2	Temperatureinstellung über Drehregler	TemperatureRotaryController	ReadInput()	TC-3-1	OK
6.2	Temperatureingabe in 1°C-Schritten	TemperatureRotaryController	ReadInput()	TC-3-1	OK
6.2	Temperaturbereich 50–300°C	TemperatureRotaryController	ReadInput()	TC-3-1	OK
6.2	Einzelsteuerung der Heizaggregate	IModeStrategy	Run()	TC-5-1	OK
6.2	Regelung mit 1 Hz	OvenController	Run()	TC-0-1	OK
6.2	Abschalten bei Solltemperatur	IModeStrategy	Run()	TC-4-2, TC-5-1	OK
6.2	Aktivieren bei Unterschreitung	IModeStrategy	Run()	TC-4-2, TC-5-1	OK
6.2	Anzeigeelemente am Display	DisplayDummy	Update()	TC-6-1	OK
6.2	200°C in unter 10 Minuten	OvenController	Run()	TC-0-2	OK

8.5 Abweichungen vom Feinentwurf

Im Zuge der Implementierung wurden einige Anpassungen gegenüber dem ursprünglichen Design vorgenommen:

- **BaseRotaryController**: Methode `GetModuloAngle()` ergänzt, um durchdrehende Regler zu unterstützen.
- **BaseRotaryController**: Eigenschaft `Angle` hinzugefügt für bessere Vererzbarkeit.
- **TemperatureRotaryController**: Einführung der Parameter `min` und `max` zur Wertebegrenzung.
- **Main** in **Program** umbenannt gemäß C#-Konventionen.
- **ModeControl** wurde in **ModeController** umbenannt.
- **Modes**: verwenden nun eine Liste von **ThermalController**-Instanzen zur gezielten Steuerung.
- **OvenController**: neue Referenz auf **InputHandler** integriert.
- **OvenController**: Methoden `GetTemperature()` und `Loop()` ergänzt.
- **DoorSensor**: neue Zustandsvariable und Methode `SetDoorState()` eingeführt.
- **TemperatureSensor**: neue Variable zur Temperaturhaltung sowie Methode `UpdateTemperature()`.
- **Ventilator**: Methode `GetTemperature()` entfernt.

9 Iteration 2 – Logging

9.1 Zeitraum

01.06.2025

9.2 Zielsetzung

Ziel dieser Iteration war die Einführung eines zentralen Logging-Mechanismus zur Laufzeitanalyse und Fehlernachverfolgung. Dabei sollte das Logging so integriert werden, dass bestehende sowie zukünftige Systemkomponenten Meldungen strukturiert und konsistent ausgeben können – ohne die eigentliche Anwendungslogik zu beeinträchtigen.

Die Logging-Funktionalität wurde exemplarisch in bereits vorhandene Module eingebunden und über manuelle Tests überprüft.

9.3 Umgesetzte Anforderungen

- **R-3.3** – Protokollierung von Systemereignissen und Fehlern mit Zeitstempel (teilweise umgesetzt; Fokus auf grundlegende Funktionalität)

9.4 Traceability-Matrix

Requirement	Beschreibung	Klasse	Methode	Testfälle	Status
R-3.3	Logging von Informationsmeldungen	Logger	Info(string)	manueller Test	OK

Die Überprüfung erfolgte manuell: Ein definierter Winkelwert wurde dem `TemperatureRotaryControl` zugewiesen. Anschließend wurde kontrolliert, ob die erwarteten Log-Ausgaben korrekt und vollständig in der Konsole erschienen. Dieses Verfahren bestätigte die erfolgreiche Integration der Logging-Funktion in das bestehende Systemverhalten.

9.5 Abweichungen vom Feinentwurf

- Entgegen dem ursprünglichen Entwurf wurde zusätzlich eine Proxy-Klasse für das Display-Modul (`DisplayDummy`) eingeführt, um auch hier eine saubere Entkopplung von Darstellung und Protokollierung zu gewährleisten.

10 Iteration 3 – Statemachine und Modestrategien

10.1 Zeitraum

02.06.2025

10.2 Zielsetzung

Ziel dieser Iteration war die Einführung einer zustandsbasierten Steuerungslogik (State-machine) sowie die Umsetzung verschiedener Heizstrategien entsprechend der gewählten Betriebsart. Dadurch wurde die Grundlage für einen dynamischen und realitätsnahen Betrieb des Systems geschaffen.

10.3 Umgesetzte Anforderungen

In dieser Iteration wurden folgende Anforderungen erfolgreich implementiert:

- **R-1.4** – Auswahl des Betriebsmodus über einen separaten Drehregler
- **R-1.5** – Sechs vordefinierte Heizmodi: Ober-/Unterhitze, Oberhitze, Unterhitze, Grill, Umluft, Heißluft
- **R-1.6** – Automatische Aktivierung der zugehörigen Heizaggregate gemäß Modusdefinition
- **R-1.11** – Umsetzung von Grillstufen durch Umrechnung der Temperatur in vier feste Stufen (240–300°C)
- **R-3.2** – Anzeige von Statusänderungen innerhalb von maximal 1 Sekunde
- **R-3.3** – Logging von Systemereignissen (erweitert)

10.4 Traceability-Matrix

Requirement	Beschreibung	Klasse	Methode	Testfälle	Status
R-1.4	Auswahl des Modus über Drehregler	ModeRotaryController	ReadInput()	TC-3-2	OK
R-1.5	Bereitstellung vordefinierter Modi	BaseModeStrategy	Run()	TC-3-2	OK
R-1.6	Aktivierung passender Heizaggregate	BaseModeStrategy	_thermalControllers	TC-4-1 bis TC-4-8	OK
R-1.11	Grillstufen-Logik (240–300°C)	GrillMode	CalculateStepTemperature()	TC-4-4	OK
R-3.2	Reaktionszeit Display < 1 Sekunde	OvenController	Run()	TC-0-1	OK
R-3.3	Logging von Statusmeldungen	Logger	Info(string)	manueller Test	OK

Die Funktionalität des Loggings wurde erneut manuell verifiziert. Dabei wurden gezielt Einstellungen am `TemperatureRotaryController` und `ModeRotaryController` vorgenommen. Die Ausgaben wurden auf Korrektheit und Vollständigkeit hin analysiert und erfüllten alle Anforderungen.

10.5 Abweichungen vom Feinentwurf

Während der Umsetzung wurden folgende Anpassungen gegenüber dem ursprünglichen Design vorgenommen:

- Einführung einer eigenen Proxy-Klasse für die Statemachine im `OvenControllerModule`, um Zustandsübergänge zu protokollieren. Dadurch entfiel die Logging-Logik im Controller selbst.
- Verzicht auf die ursprünglich geplante Enum-Klasse `OvenState`; der aktuelle Zustand wird stattdessen direkt über das aktive State-Objekt verwaltet.
- Zustandswechsel wurden in die Methode `CheckStateTransition()` innerhalb des Interfaces `IState` ausgelagert.
- States erhielten Zugriff auf notwendige Instanzvariablen zur Umsetzung ihres Verhaltens.
- Der `OffState` wurde verworfen, da seine Funktionalität durch bestehende States ausreichend abgedeckt ist.
- Einführung einer abstrakten Basisklasse `BaseModeStrategy` im `ModeControlModule`, um wiederkehrende Logik zwischen den Heizstrategien zu kapseln. Diese verwaltet auch die zugehörigen `IThermalController`-Instanzen.

11 Iteration 4 – Sicherheit und Timer

11.1 Zeitraum

02.06.2025 – 03.06.2025

11.2 Zielsetzung

Ziel dieser Iteration war die Implementierung von sicherheitsrelevanten Prüfmechanismen zur Überwachung des Systemzustands sowie die Integration eines Benutzer-Timers. Die Sicherheitsfunktionen sollen kritische Betriebszustände erkennen und entsprechende Gegenmaßnahmen automatisch einleiten. Zusätzlich wurde die Logging-Funktionalität erweitert, um auch sicherheitsrelevante Ereignisse strukturiert zu protokollieren.

11.3 Umgesetzte Anforderungen

Die folgenden Anforderungen wurden vollständig umgesetzt:

- **R-2.1** – Automatische Abschaltung bei Überhitzung ($> 320^{\circ}\text{C}$)
- **R-2.2** – Verhinderung des Heizbetriebs bei geöffneter Backofentür
- **R-2.3** – Fehlererkennung bei defektem Heizaggregat durch unzureichenden Temperaturanstieg
- **R-3.3** – Protokollierung aller sicherheitsrelevanten Zustandsänderungen mit Zeitstempel

11.4 Traceability-Matrix

Requirement	Beschreibung	Klasse	Methode	Testfälle	Status
R-2.1	Überwachung der Temperatur (Überhitzung)	OverheatRule	Check()	TC-8-6	OK
R-2.2	Türüberwachung	DoorOpenRule	Check()	TC-8-1, TC-8-2	OK
R-2.3	Heizfehler-Erkennung	HeaterFailureRule	Check()	TC-8-3 bis TC-8-5	OK
R-3.3	Logging sicherheitsrelevanter Ereignisse	Logger	Info(string)	manueller Test	OK

Zur Verifikation der Logging-Funktion wurden gezielt Szenarien erzeugt, die Sicherheitsregeln verletzen – beispielsweise Überhitzung oder Öffnen der Tür im Heizbetrieb. Die resultierenden Konsolenausgaben wurden manuell überprüft und als korrekt bewertet.

11.5 Abweichungen vom Feinentwurf

Während der Umsetzung wurden folgende Abweichungen gegenüber dem initialen Entwurf vorgenommen:

- **Kein automatisches Ausschalten über Timer:** Auf eine automatische Deaktivierung nach Ablauf des Timers wurde verzichtet. In der Praxis würde dies entweder einen motorisierten Regler erfordern oder zu inkonsistentem Systemverhalten führen. Die Timerfunktion dient ausschließlich der Anzeige.
- **Proxy-Klasse für Sicherheitsregeln:** Zur Trennung von Logging und Prüflogik wurde eine zusätzliche Proxy-Klasse für das `SafetyModule` eingeführt.
- **HeaterFailureRule:** Die Temperaturhistorie wird nun als `int`-Array anstelle von `double` gespeichert, da für die Regelung eine Ganzzahldarstellung ausreicht und Rechengenauigkeit nicht erforderlich ist.
- **OvenController-Zugriff:** Der `OvenController` wurde um eine Methode zur Abfrage des aktuellen Zustands erweitert. Ebenso wurde der entsprechende Zugriff im `StateProxy` implementiert.

12 Quellen

Die folgenden Onlinequellen wurden zur Recherche und Erstellung der funktionalen sowie technischen Anforderungen verwendet. Der Zugriff erfolgte zuletzt am **28.05.2025**.

- <https://www.hea.de/fachwissen/herde-backofen/elektroherde-aufbau-und-funktion>
- <https://www.schoener-wohnen.de/service/umluft-oder-heissluft-beim-backofen-13488.html>
- <https://de.wikipedia.org/wiki/Backofen#Haushaltsback%C3%B6fen>

13 Einsatz von Künstlicher Intelligenz

Im Rahmen dieses Projekts wurde ChatGPT-4o (OpenAI) als unterstützendes Werkzeug eingesetzt. Die KI kam in verschiedenen Phasen der Softwareentwicklung zum Einsatz – stets mit dem Ziel, eigene Ideen effizienter auszuarbeiten, zu validieren oder zu formulieren. Die finale Umsetzung und technische Verantwortung lagen dabei vollständig beim Autor.

Verwendete Einsatzbereiche

- **Requirements Engineering:** Unterstützung bei der Formulierung und Strukturierung von funktionalen und nicht-funktionalen Anforderungen.
- **Softwarearchitektur:** Diskussion und Validierung von Architekturentwürfen sowie Namensgebung der Module.
- **Softwaredesign:** Beratung zur Wahl geeigneter Entwurfsmuster (z. B. *State Pattern*, *Strategy Pattern*) und Strukturierung der Modullogik.
- **Implementierung:** Übersetzung von Klassendiagrammen in strukturierten Beispielcode (C#).
- **Softwaretest:** Unterstützung bei der Erstellung von Unit-Tests; alle generierten Tests wurden überprüft und bei Bedarf überarbeitet.
- **Dokumentation:** Formulierungshilfen und sprachliche Optimierung von Kapiteln und Abschnittsüberschriften.